

MAJOR PROJECT REPORT

On

END POINT SECURITY

Submitted to Guru Gobind Singh Indraprastha University, Delhi (India)
in partial fulfillment of the requirement for the award of the degree of

B.TECH

in

INFORMATION TECHNOLOGY

Submitted By:
Vedaang Chopra



DEPTT. OF INFORMATION TECHNOLOGY

MAHARAJASURAJMAL INSTITUTE OF TECHNOLOGY,
NEW DELHI-110058
AUGUST 2019

ACKNOWLEDGEMENT

A research work owes its success from commencement to completion, to the people in love with researchers at various stages. Let me in this page express my gratitude to all those who helped us in various stage of this study. First, I would like to express my sincere gratitude indebtedness to **Dr. Tripti Sharma** (HOD, Department of Information Technology, Maharaja Surajmal Institute of Technology, New Delhi) for allowing me to work on this project.

I am grateful to our guide Mrs. Sonika Malik for the help provided in completion of the project, which was assigned to me. Without her friendly help and guidance it was difficult to develop this project.

LIST OF CONTENTS

ACKNOWLEDGMENT	2
1. PROBLEM STATEMENT, MOTIVATION AND OBJECTIVE	7
1.1. PROBLEM STATEMENT	7
1.2. MOTIVATION	7
1.3. OBJECTIVE	7
2. PROJECT OVERVIEW	8
3. CHAPTER -1 :INTRODUCTION	9
3.1. VIRTUALIZATION	9
3.1.1. HYPERVISORS	9
3.1.2. NEED OF HYPERVISORS	9
3.2. OPERATING SYSTEMS	9
3.2.1. OPERATING SYSTEM DEFINITION	9
3.2.2. PROCESS MANAGEMENT	10
3.2.3. MEMORY MANAGEMENT	13
3.3. MALWARE ANALYSIS	16
3.3.1. USE CASE FOR MALWARE ANALYSIS	16
3.3.2. STAGES OF MALWARE ANALYSIS	17
3.3.3. STATIC ANALYSIS	18
3.3.4. DYNAMIC ANALYSIS	18
3.3.5. HYBRID ANALYSIS	18
3.4. MEMORY FORENSICS	19
3.4.1. VOLATILE DATA	19
3.4.2. IMPORTANCE	19
3.5. MACHINE LEARNING	20
3.5.1. STEPS USED IN MACHINE LEARNING	20
3.5.2. MACHINE LEARNING TYPES	21
3.5.2.1. SUPERVISED LEARNING	21
3.5.2.2. UNSUPERVISED LEARNING	22
3.5.2.3. REINFORCEMENT LEARNING	23

3.5.3. MACHINE LEARNING ALGORITHMS	24
3.5.3.1. NAÏVE BAYES	24
3.5.3.2. DECISION TREE	25
3.5.3.3. KNN	25
3.5.3.4. RANDOM FOREST	25
3.5.3.5. BAGGING	26
3.5.3.6. SVM	26
3.5.3.7. GRADIENT BOOSTING	26
4. CHAPTER-2 TECHNOLOGIES USED	27
4.1. HYPERVISORS	27
4.1.1. VMWARE WORKSTATION	27
4.1.2. VIRTUALBOX	27
4.2. CUCKOO SANDBOX	28
4.3. MEMORY FORENSICS TOOL-VOLATILITY	29
4.4. YARA	32
4.5. PYTHON	34
4.6. MACHINE LEARNING LIBRARIES	35
4.6.1. NUMPY	35
4.6.2. PANDAS	35
4.6.3. SCIKIT-LEARN	35
4.6.4. MATPLOTLIB	36
4.6.5. SEABORN	36
4.7. DEEP LEARNING LIBRARIES	37
4.7.1. KERAS	37
4.7.2. TENSORFLOW	37
5. PROJECT DESIGN	39
5.1. FLOW OF CODE	39
5.2. PHASE-1:- LAB SETUP	40
5.2.1. MALWARE ANALYSIS LAB SETUP	40
5.2.2. MEMORY FORENSICS LAB SETUP	40
5.3. PHASE-2:- SAMPLE GENERATION	41

5.3.1. MALWARE COLLECTION	41
5.3.2. MALWARE VERIFICATION	41
5.4. PHASE-3:- MEMORY ANALYSIS	42
5.4.1. CREATING MEMORY DUMPS	42
5.4.2. ANALYZING MEMORY DUMPS	42
5.4.3. YARA RULES	43
5.4.4. MODELLING RESULTS INTO A DATASET	44
5.5. PHASE-4:- APPLYING ARTIFICIAL INTELLIGENCE TO PREDICT MALWARE	45
5.5.1. DATA PRE-PROCESSING	45
5.5.2. EXPLOTARY DATA ANALYSIS	45
5.5.3. APPLYING MACHINE LEARNING ALGORTIHMS	46
5.5.4. APPLYING DEEP LEARNING ALGORTIHMS	46
6. CONCLUSION	48
6.1. MACHINE LEARNING ALGORITHMS RESULT	48
6.2. MACHINE LEARNING ALGORTIHMS COMPARISON	49
7. FUFTURE SCOPE	50
8. REFERNCES	51
9. APPENDIX-A	52

LIST OF TABLES AND FIGURES

1.	FIG-1: BASIC OPERATION SYSTEM	10
2.	FIG-2:- PROCESS IN MEMORY	11
3.	FIG-3:- PROCESS STATES	12
4.	FIG-4:- STEPS OF TEACHING A MACHINE	20
5.	FIG-5:- MACHINE LEARNING ALGORITHMS	24
6.	FIG-6:- KNN ALGORITHM	25
7.	FIG-7:- CUCKOO ARCHITECTURE	29
8.	FIG-8:- PSLIST COMMAND VOLATILITY	30
9.	FIG-9:- PSSCAN COMMAND VOLATILITY	31
10.	FIG-10:- YARASCAN COMMAND VOLATILITY	31
11.	FIG-11:- YARA RULES SYNTAX	32
12.	FIG-12:- YARA CONDITION MATCH	33
13.	FIG-13:- FLOW OF CODE OF PROJECT	39
14.	FIG-14:- MEMORY FORENSICS LAB SETUP	40
15.	FIG-15:- WANNACRY EXECUTION IN SANDBOX	41
16.	FIG-16:- IDENTIFYING BUILD PROFILE OF MEMORY DUMP	42
17.	FIG-17:- YARASCAN RESULT OF MEMORY DUMP	43
18.	FIG-18:- PROCESS OF CREATING DATASET	44
19.	FIG-19:- HEATMAP OF DATASET	45
20.	FIG-20:- DISTRIBUTION OF TARGET COLUMN	45
21.	FIG-21:-REDUCED INFORMATION PLOT OF MALWARE AND BENIGN SAMPLES	46
22.	FIG-22:- REDUCED INFORMATION PLOT OF MALWARE SAMPLES	46
23.	FIG-23:- KNN ON DATASET WITH CROSS VALIDATION	47
24.	TABLE-1:- PARAMETERS OF DIFFERENT CLASSIFIERS	48
25.	FIG-24:- COMPARING TESTING SCORE OF DATASET	49
26.	FIG-25:- COMPARING TRAINING SCORES OF DATASET	49

PROBLEM STATEMENT, MOTIVATION AND OBJECTIVE

- **PROBLEM STATEMENT:** - Even though antivirus exists, still we are vulnerable to attacks from malwares. Systems are compromised and hefty amounts need to be paid by organizations to release their data. Everyday a private user is a victim of such Malware Attacks.
- **MOTIVATION:** - While learning penetration testing, my system was compromised to demonstrate how malware (RAT) leaks out all our data without us even knowing. That led to the idea of an advanced end point security tool using Machine Learning and Deep Learning Algorithms for Pattern Recognition a basic structure was given to the project.
- **OBJECTIVE:** -Goal is to build a product that will protect these End Point Systems from Local Code Execution Attacks by enhancing the accuracy in Malware Classification and Detection strategies by studying existing strategies and developing new ones.

PROJECT OVERVIEW

Malwares have been a constant threat to every individual and organization. Even though Antivirus and End Point Security tool exists still, the number of attacks and damage cost keeps increasing exponentially. The use of signature based mapping to malwares can prevent only know malwares but zero-day malwares and obfuscated malwares can easily bypass, this technique. The need for an AI based approach is the need of hour and major antivirus providers can be seen shifting to such techniques. The technique that has been implemented in this project is analyzing malware based on their memory values. It combines the concept of **Memory Forensics, Yara Rules and Artificial Intelligence**.

CHAPTER 1: INTRODUCTION

This chapter deals with all the theoretical concepts needed to understand the background theory of this project.

1.1 Virtualization

Virtualization software allows a single host computer to create and run one or more virtual environments. Virtualization software is most often used to emulate a complete computer system in order to allow a guest operating system to be run, for example allowing Linux to run as a guest on top of a PC that is natively running a Microsoft Windows operating system (or the inverse, running Windows as a guest on Linux). Storage virtualization refers to the process of abstracting logical storage from physical storage.

HYPERVISOR

A hypervisor is software that creates and runs virtual machines (VMs). A hypervisor, sometimes called a virtual machine monitor (VMM), isolates the hypervisor operating system and resources from the virtual machines and enables the creation and management of those VMs. The physical hardware, when used as a hypervisor, is called the host, while the many VMs that use its resources are guests. The hypervisor treats resources—like CPU, memory, and storage—as a pool that can be easily reallocated between existing guests or to new virtual machines.

NEED FOR HYPERSVISORS

To create a Malware Analysis Lab, hypervisors are used to create multiple guest systems. These systems are controlled environments (sandboxes) to execute the malware.

1.2 Operating System

An Operating System (OS) is an interface between a computer user and computer hardware. An operating system is a software which performs all the basic tasks like file management, memory management, process management, handling input and output, and controlling peripheral devices such as disk drives and printers.

Some popular Operating Systems include Linux Operating System, Windows Operating System, VMS, OS/400, AIX, z/OS, etc.

DEFINITION

An operating system is a program that acts as an interface between the user and the computer hardware and controls the execution of all kinds of programs.

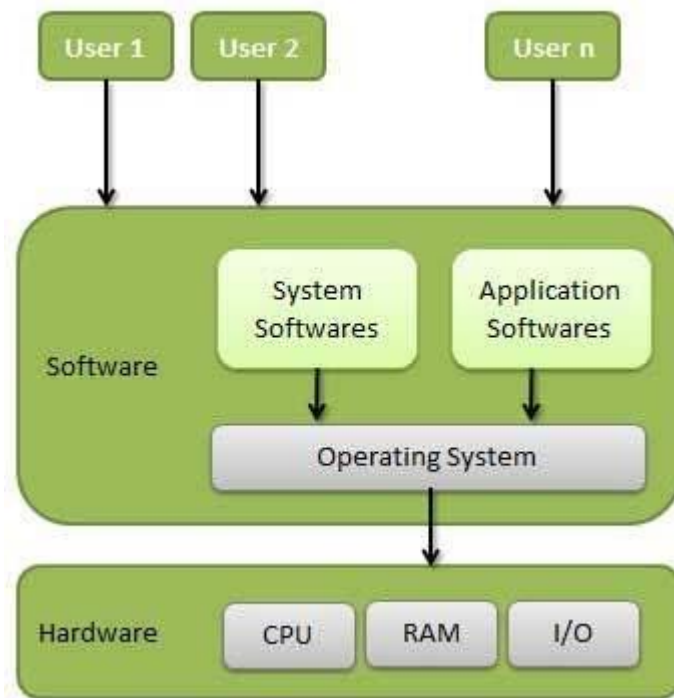


Fig 1:- Basic operating system

Following are some of important functions of an operating System.

- Memory Management
- Processor Management
- Device Management
- File Management
- Security
- Control over system performance
- Job accounting
- Error detecting aids
- Coordination between other software and users

PROCESS MANAGEMENT

A process is a program in execution. For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process.

A process is an 'active' entity, as opposed to a program, which is considered to be a 'passive' entity. A single program can create many processes when run multiple times; for example, when

we open a .exe or binary file multiple times, multiple instances begin (multiple processes are created).

What does a process look like in memory?

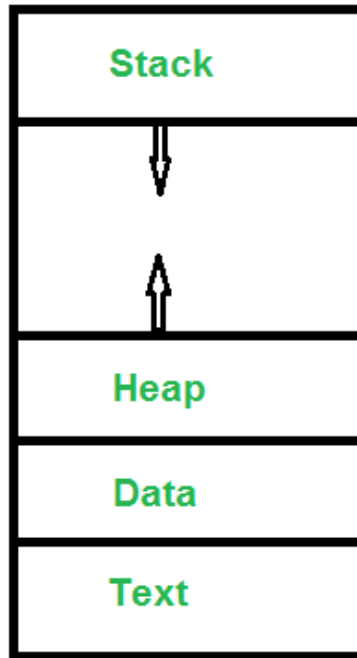


Fig 2:- Process in memory

- **Text Section:** A Process, sometimes known as the Text Section, also includes the current activity represented by the value of the Program Counter.
- **Stack:** The Stack contains the temporary data, such as function parameters, returns addresses, and local variables.
- **Data Section:** Contains the global variable.
- **Heap Section:** Dynamically allocated memory to process during its run time.

Attributes or Characteristics of a Process

A process has following attributes.

1. Process Id: A unique identifier assigned by the operating system
2. Process State: Can be ready, running, etc.
3. CPU registers: Like the Program Counter (CPU registers must be saved and restored when a process is swapped in and out of CPU)
5. Accounts information:
6. I/O status information: For example, devices allocated to the process, open files, etc

8. CPU scheduling information: For example, Priority (Different processes may have different priorities, for example a short process may be assigned a low priority in the shortest job first scheduling)

All of the above attributes of a process are also known as the context of the process. Every process has its own program control block (PCB), i.e. each process will have a unique PCB. All of the above attributes are part of the PCB.

States of Process:

A process is in one of the following states:

1. New: Newly Created Process (or) being-created process.
2. Ready: After creation process moves to Ready state, i.e. the process is ready for execution.
3. Run: Currently running process in CPU (only one process at a time can be under execution in a single processor).
4. Wait (or Block): When a process requests I/O access.
5. Complete (or Terminated): The process completed its execution.
6. Suspended Ready: When the ready queue becomes full, some processes are moved to suspended ready state
7. Suspended Block: When waiting queue becomes full.

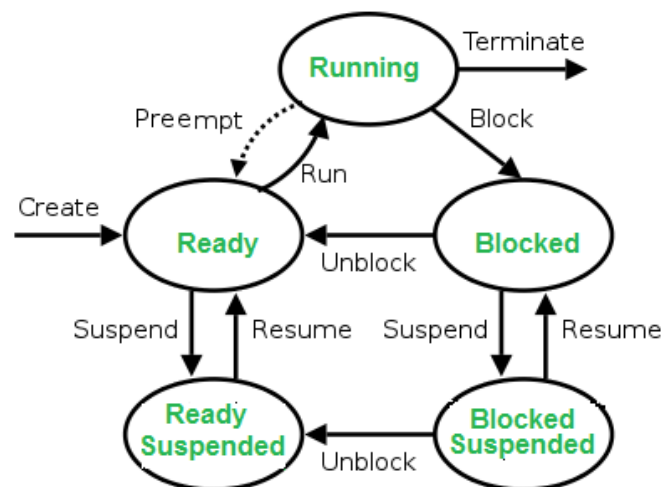


Fig 3:- Process states

Context switching

The process of saving the context of one process and loading the context of another process is known as Context Switching. In simple terms, it is like loading and unloading the process from running state to ready state.

When does context switching happen?

1. When a high-priority process comes to ready state (i.e. with higher priority than the running process)
2. An Interrupt occurs
3. User and kernel mode switch (It is not necessary though)
4. Preemptive CPU scheduling used.

Context Switch vs. Mode Switch

A mode switch occurs when CPU privilege level is changed, for example when a system call is made or a fault occurs. The kernel works in more a privileged mode than a standard user task. If a user process wants to access things which are only accessible to the kernel, a mode switch must occur. The currently executing process need not be changed during a mode switch. A mode switch typically occurs for a process context switch to occur. Only the kernel can cause a context switch.

CPU-Bound vs. I/O-Bound Processes:

A CPU-bound process requires more CPU time or spends more time in the running state. An I/O-bound process requires more I/O time and less CPU time. An I/O-bound process spends more time in the waiting state.

MEMORY MANAGEMENT

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free. It checks how much memory is to be allocated to processes. It decides which process will get memory at what time. It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

Process Address Space

The process address space is the set of logical addresses that a process references in its code. For example, when 32-bit addressing is in use, addresses can range from 0 to 0x7fffffff; that is, 2^{31} possible numbers, for a total theoretical size of 2 gigabytes.

The operating system takes care of mapping the logical addresses to physical addresses at the time of memory allocation to the program. There are three types of addresses used in a program before and after memory is allocated –

Static v/s Dynamic Loading

The choice between Static or Dynamic Loading is to be made at the time of computer program being developed. If you have to load your program statically, then at the time of compilation, the complete programs will be compiled and linked without leaving any external program or module dependency. The linker combines the object program with other necessary object modules into an absolute program, which also includes logical addresses.

If you are writing a dynamically loaded program, then your compiler will compile the program and for all the modules which you want to include dynamically, only references will be provided and rest of the work will be done at the time of execution.

At the time of loading, with static loading, the absolute program (and data) is loaded into memory in order for execution to start.

If you are using dynamic loading, dynamic routines of the library are stored on a disk in relocatable form and are loaded into memory only when they are needed by the program.

Swapping

Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes. At some later time, the system swaps back the process from the secondary storage to main memory.

Though performance is usually affected by swapping process but it helps in running multiple and big processes in parallel and that's the reason Swapping is also known as a technique for memory compaction.

The total time taken by swapping process includes the time it takes to move the entire process to a secondary disk and then to copy the process back to memory, as well as the time the process takes to regain main memory.

Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into little pieces. It happens after sometimes that processes cannot be allocated to memory blocks considering their small size and memory blocks remains unused. This problem is known as Fragmentation.

Fragmentation is of two types –

The following diagram shows how fragmentation can cause waste of memory and a compaction technique can be used to create more free memory out of fragmented memory –

External fragmentation can be reduced by compaction or shuffle memory contents to place all free memory together in one large block. To make compaction feasible, relocation should be dynamic.

The internal fragmentation can be reduced by effectively assigning the smallest partition but large enough for the process.

Paging

A computer can address more memory than the amount physically installed on the system. This extra memory is actually called virtual memory and it is a section of a hard that's set up to emulate the computer's RAM. Paging technique plays an important role in implementing virtual memory.

Paging is a memory management technique in which process address space is broken into blocks of the same size called pages (size is power of 2, between 512 bytes and 8192 bytes). The size of the process is measured in the number of pages.

Similarly, main memory is divided into small fixed-sized blocks of (physical) memory called frames and the size of a frame is kept the same as that of a page to have optimum utilization of the main memory and to avoid external fragmentation.

Advantages and Disadvantages of Paging

Here is a list of advantages and disadvantages of paging –

- Paging reduces external fragmentation, but still suffer from internal fragmentation.
- Paging is simple to implement and assumed as an efficient memory management technique.
- Due to equal size of the pages and frames, swapping becomes very easy.
- Page table requires extra memory space, so may not be good for a system having small RAM

Segmentation

Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.

When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory. Segmentation memory management works very similar to paging but here segments are of variable-length where as in paging pages are of fixed size.

A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a segment map table for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory. For each segment, the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.

1.3 Malware Analysis

Malware analysis is the process of learning how malware functions and any potential repercussions of a given malware. Malware code can differ radically, and it's essential to know that malware can have many functionalities. These may come in the form of viruses, worms, spyware, and Trojan horses. Each type of malware gathers information about the infected device without the knowledge, or authorization of the user.

USE CASES FOR MALWARE ANALYSIS

- Computer security incident management: If an organization believes that malware may have entered into its system, a response team will react to the situation. Next, they will want to perform malware analysis on any potentially malicious files that are discovered.

This will then determine if it is indeed malware, what type, and the impact that it might have on the respective organizations' systems.

- **Malware research:** Academic or industry forum where malware researchers perform malware analysis. This creates the best understanding of how malware works and the newest methods used in its creation.
- **Indicator of compromise (IOC) extraction:** Sellers of software solutions and products may conduct bulk malware analysis in order to determine potential new indicators of compromise which will in turn help the organizations to defend themselves against malware attacks.

FOUR STAGES OF MALWARE ANALYSIS

Investigating malware is a process that requires taking a few steps. These four stages form a pyramid that grows in intricacy. The closer you get to the top of the pyramid, the stages increase in complexity and the skills needed to implement them are less common. Here, we start from the bottom, and show you what goes into finding malware, every step of the way.

- **Fully-automated analysis:** One of the simplest ways to assess a suspicious program is to scan it with fully-automated tools. Fully-automated tools are able to quickly assess what a malware is capable of if it infiltrated the system. This analysis is able to produce a detailed report regarding the network traffic, file activity, and registry keys. Even though a fully-automated analysis does not provide as much information as an analyst, it is still the fastest method to sift through large quantities of malware.
- **Static properties analysis:** In order to get a more in depth look at malware, it is imperative to look at its static properties. It is easy to access these properties because it does not require running the potential malware, which takes a longer time. The static properties include hashes, embedded strings, embedded resources, and header information. The properties should be able to show elementary indicators of compromise.
- **Interactive behavior analysis:** To observe a malicious file, it might often times be put in an isolated laboratory to see if it directly infects the laboratory. Analysts will frequently monitor these laboratories to see if the malicious file tries to attach to any hosts. With this information, the analyst will then be able to replicate the situation to see what the malicious file would do once it was connected to the host, giving them an advantage over those who use automated tools.
- **Manual code reversing:** Reversing the code of the malicious file can decode encrypted data that was stored by the sample, determine the logic of the file's domain, and see other capabilities of the file that did not show up during the behavioral analysis. In order to manually reverse the code, malware analysis tools such as a debugger and disassembler are needed. The skills needed to complete manual code reversing are very important, but also difficult to find.

Static Analysis

Basic static analysis does not require that the code is actually run. Instead, static analysis examines the file for signs of malicious intent. It can be useful to identify malicious infrastructure, libraries or packed files. Technical indicators are identified such as file names, hashes, strings such as IP addresses, domains, and file header data can be used to determine whether that file is malicious. In addition, tools like disassemblers and network analyzers can be used to observe the malware without actually running it in order to collect information on how the malware works. However, since static analysis does not actually run the code, sophisticated malware can include malicious runtime behavior that can go undetected. For example, if a file generates a string that then downloads a malicious file based upon the dynamic string, it could go undetected by a basic static analysis. Enterprises have turned to dynamic analysis for a more complete understanding of the behavior of the file.

Dynamic Analysis

Dynamic malware analysis executes suspected malicious code in a safe environment called a sandbox. This closed system enables security professionals to watch the malware in action without the risk of letting it infect their system or escape into the enterprise network.

Dynamic analysis provides threat hunters and incident responders with deeper visibility, allowing them to uncover the true nature of a threat. As a secondary benefit, automated sandboxing eliminates the time it would take to reverse engineer a file to discover the malicious code.

The challenge with dynamic analysis is that adversaries are smart, and they know sandboxes are out there, so they have become very good at detecting them. To deceive a sandbox, adversaries hide code inside them that may remain dormant until certain conditions are met. Only then does the code run.

Hybrid Analysis

Basic static analysis isn't a reliable way to detect sophisticated malicious code, and sophisticated malware can sometimes hide from the presence of sandbox technology. By combining basic and dynamic analysis techniques, hybrid analysis provide security team the best of both approaches – primarily because it can detect malicious code that is trying to hide, and then can extract many more indicators of compromise (IOCs) by statically and previously unseen code. Hybrid analysis helps detect unknown threats, even those from the most sophisticated malware.

For example, one of the things hybrid analysis does is apply static analysis to data generated by behavioral analysis – like when a piece of malicious code runs and generates some changes in memory. Dynamic analysis would detect that, and analysts would be alerted to circle back and perform basic static analysis on that memory dump. As a result, more IOCs would be generated and zero-day exploits would be exposed.

1.4 Memory Forensics

Memory forensics (sometimes referred to as memory analysis) refers to the analysis of volatile data in a computer's memory dump. Information security professionals conduct memory forensics to investigate and identify attacks or malicious behaviors that do not leave easily detectable tracks on hard drive data.

WHAT IS VOLATILE DATA?

Volatile data is the data stored in temporary memory on a computer while it is running. When a computer is powered off, volatile data is lost almost immediately. Volatile data resides in a computer's short term memory storage and can include data like browsing history, chat messages, and clipboard contents. If, for example, you were working on a document in Word or Pages that you had not yet saved to your hard drive or another non-volatile memory source, then you would lose your work if your computer lost power before it was saved.

A memory dump (also known as a core dump or system dump) is a snapshot capture of computer memory data from a specific instant. A memory dump can contain valuable forensics data about the state of the system before an incident such as a crash or security compromise. Memory dumps contain RAM data that can be used to identify the cause of an incident and other key details about what happened.

THE IMPORTANCE OF MEMORY FORENSICS

Memory forensics can provide unique insights into runtime system activity, including open network connections and recently executed commands or processes. In many cases, critical data pertaining to attacks or threats will exist solely in system memory – examples include network connections, account credentials, chat messages, encryption keys, running processes, injected code fragments, and internet history which is non-cacheable. Any program – malicious or otherwise – must be loaded in memory in order to execute, making memory forensics critical for identifying otherwise obfuscated attacks.

As attack methods become increasingly sophisticated, memory forensics tools and skills are in high demand for security professionals today. Many network-based security solutions like firewalls and antivirus tools are unable to detect malware written directly into a computer's physical memory or RAM. Security teams should look to memory forensics tools and specialists

to protect invaluable business intelligence and data from stealthy attacks such as fileless, in-memory malware or RAM scrapers.

1.5 MACHINE LEARNING AND DEEP LEARNING

Teaching the machines involve a structural process where every stage builds a better version of the machine. For simplification purpose, the process of teaching machines can be broken down into 3 parts:

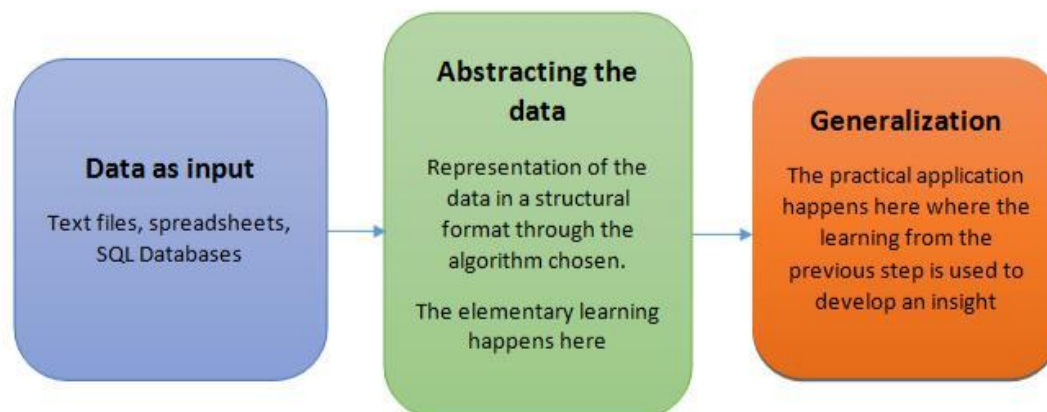


Fig 1: Steps of teaching a machine

These 3 steps ensure the holistic learning of the machine to perform the given task with equal importance. Success of machine depends on two factors:

How well the generalization of abstraction data takes place.

How well the machine is able to put it's learning into practical usage for predicting the future course of action.

STEPS USED IN MACHINE LEARNING

There are 5 basic steps used to perform a machine learning task:

- Collecting data: Be it the raw data from excel, access, text files etc., this step (gathering past data) forms the foundation of the future learning. The better the variety, density and volume of relevant data, better the learning prospects for the machine becomes.
- Preparing the data: Any analytical process thrives on the quality of the data used. One needs to spend time determining the quality of data and then taking steps for fixing issues

such as missing data and treatment of outliers. Exploratory analysis is perhaps one method to study the nuances of the data in details thereby burgeoning the nutritional content of the data.

- Training a model: This step involves choosing the appropriate algorithm and representation of data in the form of the model. The cleaned data is split into two parts – train and test (proportion depending on the prerequisites); the first part (training data) is used for developing the model. The second part (test data), is used as a reference.
- Evaluating the model: To test the accuracy, the second part of the data (holdout / test data) is used. This step determines the precision in the choice of the algorithm based on the outcome. A better test to check accuracy of model is to see its performance on data which was not used at all during model build.
- Improving the performance: This step might involve choosing a different model altogether or introducing more variables to augment the efficiency. That's why significant amount of time needs to be spent in data collection and preparation.

Be it any model, these 5 steps can be used to structure the technique.

TYPES OF ML ALGORITHMS

SUPERVISED LEARNING

It is also called predictive model. As the name suggests is used to predict the future outcome based on the historical data. Predictive models are normally given clear instructions right from the beginning as in what needs to be learnt and how it needs to be learnt. These classes of learning algorithms are termed as Supervised Learning.

For example: Supervised Learning is used when a marketing company is trying to find out which customers are likely to churn.

We can also use it to predict the likelihood of occurrence of perils like earthquakes, tornadoes etc. with an aim to determine the Total Insurance Value. Some examples of algorithms used are: Nearest neighbor, Naïve Bayes, Decision Trees, and Regression etc.

The majority of practical machine learning uses supervised learning.

Supervised learning is where you have input variables (x) and an output variable (Y) and you use an algorithm to learn the mapping function from the input to the output. $Y = f(X)$

The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.

It is called supervised learning because the process of algorithm learning from the training dataset can be thought of as a teacher supervising the learning process. We know the correct answers; the algorithm iteratively makes predictions on the training data and is corrected by the teacher. Learning stops when the algorithm achieves an acceptable level of performance.

Supervised learning problems can be further grouped into regression and classification problems. Some common types of problems built on top of classification and regression include recommendation and time series prediction respectively.

Some popular examples of supervised machine learning algorithms are:

- Linear regression for regression problems.
- Random forest for classification and regression problems.
- Support vector machines for classification problems.

The supervised learning problem can be either:

- **Classification:** samples belong to two or more classes and we want to learn from already labeled data how to predict the class of unlabeled data. An example of classification problem would be the handwritten digit recognition example, in which the aim is to assign each input vector to one of a finite number of discrete categories. Another way to think of classification is as a discrete (as opposed to continuous) form of supervised learning where one has a limited number of categories and for each of the n samples provided, one is to try to label them with the correct category or class.
- **Regression:** if the desired output consists of one or more continuous variables, then the task is called *regression*. An example of a regression problem would be the prediction of the length of a salmon as a function of its age and weight.

UNSUPERVISED LEARNING

Unsupervised learning is used to train descriptive models where no target is set and no single feature is important than the other. The case of unsupervised learning can be: When a retailer wishes to find out what are the combination of products, customers tends to buy more frequently. Furthermore, in pharmaceutical industry, unsupervised learning may be used to predict which diseases are likely to occur along with diabetes. Example of algorithm used here is: K- means Clustering Algorithm.

Unsupervised learning is where you only have input data (X) and no corresponding output variables. The goal for unsupervised learning is to model the underlying structure or distribution

in the data in order to learn more about the data. These are called unsupervised learning because unlike supervised learning above there are no correct answers and there is no teacher. Algorithms are left to their own devices to discover and present the interesting structure in the data.

Unsupervised learning problems can be further grouped into clustering and association problems.

- **Clustering:** A clustering problem is where you want to discover the inherent groupings in the data, such as grouping customers by purchasing behavior.
- **Association:** An association rule learning problem is where you want to discover rules that describe large portions of your data, such as people that buy X also tend to buy Y.

Some popular examples of unsupervised learning algorithms are:

- K-means for clustering problems.
- Apriori algorithm for association rule learning problems.

The goal in such problems may be to discover groups of similar examples within the data, where it is called clustering, or to determine the distribution of data within the input space, known as density estimation, or to project the data from a high-dimensional space down to two or three dimensions for the purpose of visualization.

REINFORCEMENT LEARNING

It is an example of machine learning where the machine is trained to take specific decisions based on the business requirement with the sole motto to maximize efficiency (performance). The idea involved in reinforcement learning is: The machine/ software agent trains itself on a continual basis based on the environment it is exposed to, and applies its enriched knowledge to solve business problems. This continual learning process ensures less involvement of human expertise which in turn saves a lot of time! An example of algorithm used in RL is Markov Decision Process.

There is a subtle difference between Supervised Learning and Reinforcement Learning (RL). RL essentially involves learning by interacting with an environment. An RL agent learns from its past experience, rather from its continual trial and error learning process as against supervised learning where an external supervisor provides examples.

A good example to understand the difference is self-driving cars. Self-driving cars use Reinforcement learning to make decisions continuously – which route to take? what speed to drive on? are some of the questions which are decided after interacting with the environment. A

simple manifestation for supervised learning would be to predict fare from a cab going from one place to another.

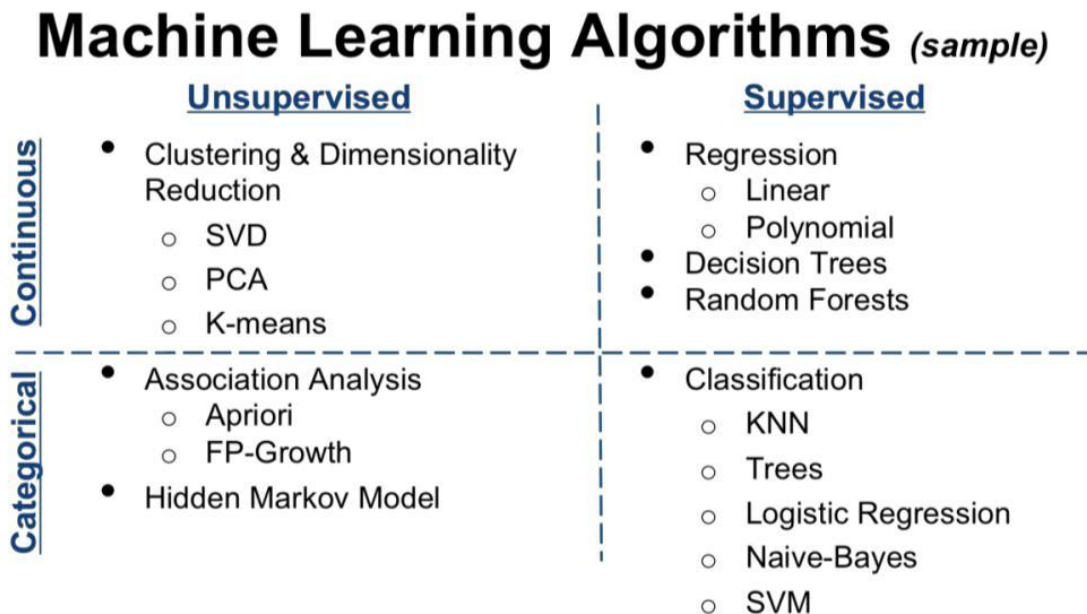


Fig 5: Machine Learning Algorithms

MACHINE LEARNING ALGORITHMS

- **Naïve Bayes classifier**- It uses the Bayes theorem to predict values. This algorithm is good for multi-class classification.
 - Consider a data point x , in a multi-class scenario with three classes- A, B and C. Using Naïve Bayes, we try to predict whether the data point x belongs to class A or B or C, by calculating its probability for the three classes.
 - $P(A/B) = (P(B/A).P(A))/P(B)$
 - This algorithm is called 'Naïve', because it assumes that all the features are independent of each other.
 - $P(f_1, f_2, f_3, \dots, f_n) = P(f_1) = P(f_2) = \dots = P(f_n)$
 - There are further two categories of the NB Classifier, the Gaussian Naïve Bayes Classifier, and the multinomial Naïve-Bayes Classifier.
 - The Gaussian Naïve-Bayes classifier is used when a dataset has continuous values of data. It uses the Gaussian Probability Distribution function (values are centered around mean and as the graph grows, the values decrease).

- **Multinomial Naive Bayes:** The Naïve Bayes algorithm assumes the independence of features, and the multinomial component of this classifier ensures that the distribution is multinomial in its features.
- **The Nearest Centroid classifier** is a simple algorithm that represents each class by the centroid of its members. The centroid for the samples corresponding to each class is the point from which the sums of the distances of all samples that belong to that particular class are minimized.
- **Decision Tree-** It is a highly intuitive algorithm which uses greedy approach. To construct a decision tree, we have to perform the following steps – (i) select a feature to split the data, (ii) select a method to split the data on the said feature. It has the internal working algorithm as:
 - Create/Select a node.
 - If the node is pure, output the only class
 - If no feature is left to split upon, and the node is impure, output the majority class.
 - Else find the best feature to split upon. Recursively call on this split. Goto step (ii)
- **KNN** - Consider a scenario where we have to predict to which class, the testing point belongs to, by considering all the features at once. Such is the working of KNN algorithm. To predict the class of the testing data point, we check its vicinity.

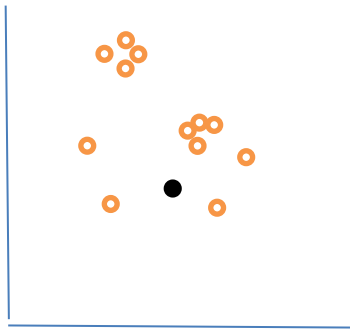


Fig 6: - KNN Algorithm

To classify the testing point, we check a specific number of points (1, 3, 5, 7, etc.) and whichever class is in majority among those, that one is predicted. To select the nearest point, we have to consider its distance from the other points. The distance metric can be (a) Manhattan distance, (b) Euclidian Distance, (c) Makowski distance.

(f) **Random forest** - It is an extension of the decision tree classifier. This algorithm uses multiple combinations of decision trees to accurately predict testing data. The random forest

classifier overcomes the overfitting problem of decision trees by building multiple decision trees and going with the majority result. The trees' outputs vary because each tree is built with random data and random features. To generate randomness in trees, we use two techniques-

- (i) **Bagging:** If we have 'm' data points, we select a subset of 'k' out of them. For 'n' trees, n*k subsets are selected. Data points can be considered with replacement as the selection is random, therefore, these trees are called bag trees.
- (ii) **Feature Selection:** In the training phase, some features are selected at random in this technique, with the condition that the selection is performed without replacement.

(g) **SVM Classifier-** It is a very powerful algorithm and overcomes the limitation of logistic regression. As logistic regression uses sigmoid function, the value predicted for a testing data point is close to 0.5. This causes the problem of incorrect prediction. So, SVM uses the rules of logistic regression only, but exponentially increases the value, so that the values predicted do not fall in the range (-1, 1). The cost function of the logistic regression is:

$$E(h(x)) = \sum (-y_i \log(h(x)) - (1-y_i) \log(1-h(x)))$$

This cost function changes to the following equation in SVM –

$$(\theta) = C \sum [y(i) \text{cost}_1(\theta T x(i)) + (1-y(i) \text{cost}_0(\theta T x(i)))] + 0.5 \sum (\theta_i)^2$$

(h) **Gradient Boosting Algorithm-** It continuously creates models that are nothing but improvisation on the previous version. It essentially keeps strengthening the model until a model strong enough for correct prediction is generated. This is done by building a model from the training data, then creating a second model that attempts to correct the errors from the first model. The addition of improved models terminates when either of the two outcomes are achieved – (i) the prediction is perfected or (ii) the upper limit for adding models is reached

Deep learning is a particular kind of machine learning that achieves great power and flexibility by learning to represent the world as nested hierarchy of concepts, with each concept defined in relation to simpler concepts, and more abstract representations computed in terms of less abstract ones.

CHAPTER 2: ADDITIONAL TOOLS USED

2.1 Hypervisor

2.1.1 VMware Workstation Player

VMware Workstation Player (formerly known as Player Pro) is a desktop virtualization application that is available for free for personal use. Use VMware Workstation Player to create, run, evaluate, and share software running in virtual machines:

- **Create:** Use VMware Workstation Player to create virtual machines with the latest 32-bit and 64-bit Windows and Linux operating systems. With Easy Install it's easier than installing them directly on your PC.
- **Run:** VMware Workstation Player can be used by anyone to run virtual machines on a Windows or Linux PC. VMware Workstation Player makes it quick and easy to take advantage of the security, flexibility, and portability of virtual machines.

2.1.2 VirtualBox

VirtualBox is a powerful x86 and AMD64/Intel64 virtualization product for enterprise as well as home use. Not only is VirtualBox an extremely feature rich, high performance product for enterprise customers, it is also the only professional solution that is freely available as Open Source Software under the terms of the GNU General Public License (GPL) version 2. Presently, VirtualBox runs on Windows, Linux, Macintosh, and Solaris hosts and supports a large number of guest operating systems including but not limited to Windows (NT 4.0, 2000, XP, Server 2003, Vista, Windows 7, Windows 8, Windows 10), DOS/Windows 3.x, Linux (2.4, 2.6, 3.x and 4.x), Solaris and OpenSolaris, OS/2, and OpenBSD.

In order to run VirtualBox on your machine, you need:

- **Reasonably powerful x86 hardware.** Any recent Intel or AMD processor should do.
- **Memory:** Depending on what guest operating systems you want to run, you will need at least 512 MB of RAM (but probably more, and the more the better). Basically, you will need whatever your host operating system needs to run comfortably, plus the amount that the guest operating system needs.
- **Hard disk space.** While VirtualBox itself is very lean (a typical installation will only need about 30 MB of hard disk space), the virtual machines will require fairly huge files on disk to represent their own hard disk storage. So, to install Windows 8, for example, you will need a file that will easily grow to several 10 GB in size.

- **A supported host operating system.** Presently, we support Windows, many Linux distributions, Mac OS X, Solaris and OpenSolaris. Check the user manual of the VirtualBox version you are using which versions are supported. For the latest VirtualBox version, see manual.
- **A supported guest operating system.** Besides the user manual (see below), up-to-date information is available at "Status: Guest OSes".

2.2 Cuckoo Sandbox

Cuckoo is an open source automated malware analysis system.

It's used to automatically run and analyze files and collect comprehensive analysis results that outline what the malware does while running inside an isolated operating system.

It can retrieve the following type of results:

- Traces of calls performed by all processes spawned by the malware.
- Files being created, deleted and downloaded by the malware during its execution.
- Memory dumps of the malware processes.
- Network traffic trace in PCAP format.
- Screenshots taken during the execution of the malware.
- Full memory dumps of the machines.

Cuckoo is designed to be used both as a standalone application as well as to be integrated in larger frameworks, thanks to its extremely modular design.

It can be used to analyze:

- Generic Windows executable
- DLL files
- PDF documents
- Microsoft Office documents
- URLs and HTML files
- PHP scripts
- CPL files
- Visual Basic (VB) scripts
- ZIP files
- Java JAR
- Python files
- Almost anything else

Architecture

Cuckoo Sandbox consists of a central management software which handles sample execution and analysis.

Each analysis is launched in a fresh and isolated virtual or physical machine. The main components of Cuckoo's infrastructure are an Host machine (the management software) and a number of Guest machines (virtual or physical machines for analysis).

The Host runs the core component of the sandbox that manages the whole analysis process, while the Guests are the isolated environments where the malware samples get actually safely executed and analyzed.

The following picture explains Cuckoo's main architecture:

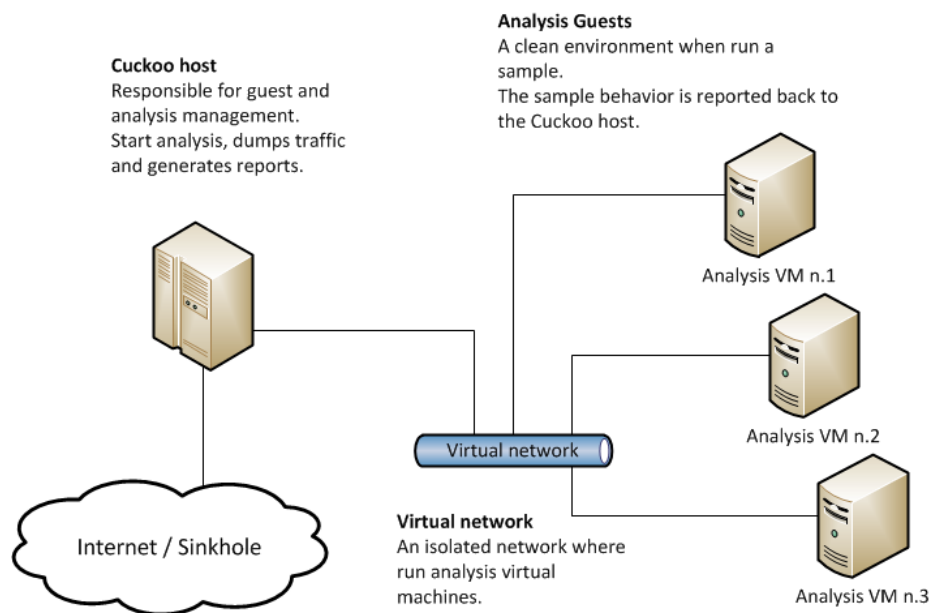


Fig 7: - Cuckoo Architecture

Thanks to its modularity and powerful scripting capabilities, there's no limit to what you can achieve with Cuckoo.

Installation: - To Install Cuckoo follow the documentation on the following link:
<https://cuckoo.sh/docs/installation/index.html>

2.3 Volatility

The Volatility Framework is a completely open collection of tools, implemented in Python under the GNU General Public License, for the extraction of digital artifacts from volatile memory (RAM) samples. The extraction techniques are performed completely independent of the system

being investigated but offer visibility into the runtime state of the system. The framework is intended to introduce people to the techniques and complexities associated with extracting digital artifacts from volatile memory samples and provide a platform for further work into this exciting area of research.

Volatility Commands:

- **pslist**

To list the processes of a system, use the pslist command. This walks the doubly-linked list pointed to by PsActiveProcessHead and shows the offset, process name, process ID, the parent process ID, number of threads, number of handles, and date/time when the process started and exited. As of 2.1 it also shows the Session ID and if the process is a Wow64 process (it uses a 32 bit address space on a 64 bit kernel).

```
$ python vol.py -f ~/Desktop/win7_trial_64bit.raw --profile=Win7SP0x64 pslist
```

Volatility Foundation Volatility Framework 2.4

Offset(V)	Name	PID	PPID	Thds	Hnds	Sess	Wow64	Start
0xfffffa80004b09e0	System	4	0	78	489	-----	0	2012-02-22
0xfffffa8000ce97f0	smss.exe	208	4	2	29	-----	0	2012-02-22
0xfffffa8000c006c0	csrss.exe	296	288	9	385	0	0	2012-02-22
0xfffffa8000c92300	wininit.exe	332	288	3	74	0	0	2012-02-22
0xfffffa8000c06b30	csrss.exe	344	324	7	252	1	0	2012-02-22
0xfffffa8000c80b30	winlogon.exe	372	324	5	136	1	0	2012-02-22
0xfffffa8000c5eb30	services.exe	428	332	6	193	0	0	2012-02-22
0xfffffa80011c5700	lsass.exe	444	332	6	557	0	0	2012-02-22
0xfffffa8000ea31b0	lsm.exe	452	332	10	133	0	0	2012-02-22
0xfffffa8001296b30	svchost.exe	568	428	10	352	0	0	2012-02-22
0xfffffa80012c3620	svchost.exe	628	428	6	247	0	0	2012-02-22
0xfffffa8001325950	sppsvc.exe	816	428	5	154	0	0	2012-02-22
0xfffffa80007b7960	svchost.exe	856	428	16	404	0	0	2012-02-22
0xfffffa80007bb750	svchost.exe	880	428	34	1118	0	0	2012-02-22
0xfffffa80007d09e0	svchost.exe	916	428	19	443	0	0	2012-02-22
0xfffffa8000c64840	svchost.exe	348	428	14	338	0	0	2012-02-22
0xfffffa8000c09630	svchost.exe	504	428	16	496	0	0	2012-02-22
0xfffffa8000e86690	spoolsv.exe	1076	428	12	271	0	0	2012-02-22
0xfffffa8000518b30	svchost.exe	1104	428	18	307	0	0	2012-02-22
0xfffffa800094d960	wlms.exe	1264	428	4	43	0	0	2012-02-22
0xfffffa8000995b30	svchost.exe	1736	428	12	200	0	0	2012-02-22
0xfffffa8000aa0b30	SearchIndexer.	1800	428	12	757	0	0	2012-02-22
0xfffffa8000aea630	taskhost.exe	1144	428	7	189	1	0	2012-02-22
0xfffffa8000eafb30	dwm.exe	1476	856	3	71	1	0	2012-02-22
0xfffffa80008f3420	explorer.exe	1652	840	21	760	1	0	2012-02-22
0xfffffa8000c9a630	regsvr32.exe	1180	1652	0	-----	1	0	2012-02-22
0xfffffa8000a03b30	rundll32.exe	2016	568	3	67	1	0	2012-02-22
0xfffffa8000a4f630	svchost.exe	1432	428	12	350	0	0	2012-02-22
0xfffffa8000999780	iexplore.exe	1892	1652	19	688	1	1	2012-02-22
0xfffffa80010c9060	iexplore.exe	2820	1892	23	733	1	1	2012-02-22
0xfffffa8001016060	DumpIt.exe	2860	1652	2	42	1	1	2012-02-22
0xfffffa8000acab30	conhost.exe	2236	344	2	51	1	0	2012-02-22

Fig 8: - pslist Command Volatility

- **psscan**

To enumerate processes using pool tag scanning (_POOL_HEADER), use the psscan command. This can find processes that previously terminated (inactive) and processes that have been hidden or unlinked by a rootkit. The downside is that rootkits can still hide by overwriting the pool tag values (though not commonly seen in the wild).

```
$ python vol.py --profile=Win7SP0x86 -f win7.dmp psscan
Volatility Foundation Volatility Framework 2.0
```

Offset	Name	PID	PPID	PDB	Time created	Time exited
0x3e025ba8	svchost.exe	1116	508	0x3ecf1220	2010-06-16 15:25:25	
0x3e04f070	svchost.exe	1152	508	0x3ecf1340	2010-06-16 15:27:40	
0x3e144c08	dwm.exe	1540	832	0x3ecf12e0	2010-06-16 15:26:58	
0x3e145c18	TPAutoConnSvc.	1900	508	0x3ecf1360	2010-06-16 15:25:41	
0x3e3393f8	lsass.exe	516	392	0x3ecf10e0	2010-06-16 15:25:18	
0x3e35b8f8	svchost.exe	628	508	0x3ecf1120	2010-06-16 15:25:19	
0x3e383770	svchost.exe	832	508	0x3ecf11a0	2010-06-16 15:25:20	
0x3e3949d0	svchost.exe	740	508	0x3ecf1160	2010-06-16 15:25:20	
0x3e3a5100	svchost.exe	872	508	0x3ecf11c0	2010-06-16 15:25:20	
0x3e3f64e8	svchost.exe	992	508	0x3ecf1200	2010-06-16 15:25:24	
0x3e45a530	wininit.exe	392	316	0x3ecf10a0	2010-06-16 15:25:15	
0x3e45d928	svchost.exe	1304	508	0x3ecf1260	2010-06-16 15:25:28	
0x3e45f530	csrss.exe	400	384	0x3ecf1040	2010-06-16 15:25:15	
0x3e4d89c8	vmtoolsd.exe	1436	508	0x3ecf1280	2010-06-16 15:25:30	
0x3e4db030	spoolsv.exe	1268	508	0x3ecf1240	2010-06-16 15:25:28	
0x3e50b318	services.exe	508	392	0x3ecf1080	2010-06-16 15:25:18	
0x3e7f3d40	csrss.exe	352	316	0x3ecf1060	2010-06-16 15:25:12	
0x3e7f5bc0	winlogon.exe	464	384	0x3ecf10c0	2010-06-16 15:25:18	
0x3eac6030	SearchProtocol	2448	1168	0x3ecf15c0	2010-06-16 23:30:52	2010-06-16 23:33
0x3eb10030	SearchFilterHo	1812	1168	0x3ecf1480	2010-06-16 23:31:02	2010-06-16 23:33

```
[snip]
```

Fig 9:- psscan Command Volatility

- yarascan

Volatility has several built-in scanning engines to help you find simple patterns like pool tags in physical or virtual address spaces. However, if you need to scan for more complex things like regular expressions or compound rules (i.e. search for "this" and not "that"), you can use the yarascan command. This plugin can help you locate any sequence of bytes (like assembly instructions with wild cards), regular expressions, ANSI strings, or Unicode strings in user mode or kernel memory.

```
$ python vol.py -f td13.vmem yarascan --yara-rules="{8B 00 81 38 54 44 4C 33 75 5A}" -K
Volatility Foundation Volatility Framework 2.4
Rule: r1
Owner: (Unknown Kernel Memory)
0x8138dcc0 8b 00 81 38 54 44 4c 33 75 5a 8b 45 f4 05 fd 29 ...8TDL3uZ.E...)
0x8138dcd0 b7 f0 50 b8 08 03 00 00 8b 80 00 00 df ff ff b0 ..P.....
0x8138dce0 00 01 00 00 b8 08 03 00 00 8b 80 00 00 df ff 8b .....
0x8138dcf0 40 04 8b 4d ec 03 41 20 ff d0 ff 75 e0 b8 08 03 @..M..A....u....
Rule: r1
Owner: dump_vmcs.sys
0xf94bb4c3 8b 00 81 38 54 44 4c 33 75 5a 8b 45 f4 05 fd 29 ...8TDL3uZ.E...)
0xf94bb4d3 b7 f0 50 b8 08 03 00 00 8b 80 00 00 df ff ff b0 ..P.....
0xf94bb4e3 00 01 00 00 b8 08 03 00 00 8b 80 00 00 df ff 8b .....
0xf94bb4f3 40 04 8b 4d ec 03 41 20 ff d0 ff 75 e0 b8 08 03 @..M..A....u....
Rule: r1
Owner: vmcs.sys
0xf9dba4c3 8b 00 81 38 54 44 4c 33 75 5a 8b 45 f4 05 fd 29 ...8TDL3uZ.E...)
0xf9dba4d3 b7 f0 50 b8 08 03 00 00 8b 80 00 00 df ff ff b0 ..P.....
0xf9dba4e3 00 01 00 00 b8 08 03 00 00 8b 80 00 00 df ff 8b .....
0xf9dba4f3 40 04 8b 4d ec 03 41 20 ff d0 ff 75 e0 b8 08 03 @..M..A....u....
```

Fig 10:- yarascan Command Volatility

2.4 Yara

YARA rules are a way of identifying malware (or other files) by creating rules that look for certain characteristics. YARA was originally developed by Victor Alvarez of Virustotal and is mainly used in malware research and detection. It was developed with the idea to describe patterns that identify particular strains or entire families of malware.

Syntax

Each rule has to start with the word *rule*, followed by the name or identifier. The identifier can contain any alphanumeric character and the underscore character, but the first character is not allowed to be a digit. There is a list of YARA keywords that are not allowed to be used as an identifier because they have a predefined meaning.

```
rule rogues
{
    meta:
        created   = "09/11/2017 00:00:00"
        modified  = "09/11/2017 17:12:07"
        author    = "Metallica"
        Vendor    = "PC Smart Cleanup"
    strings:
        $textstring1 = "smart"  ascii wide nocase
        $textstring2 = "cleanup" ascii wide nocase
        $textstring3 = "longrun" ascii wide nocase
    condition:
        $textstring1 and @textstring2 and $textstring3
}
```

Fig 11: - Yara Rules Syntax

Condition

Rules are composed of several sections. The *condition* section is the only one that is required. This section specifies when the rule result is *true* for the object (file) that is under investigation. It contains a Boolean expression that determines the result. Conditions are by design Boolean expressions and can contain all the usual logical and relational operators. You can also include another rule as part of your conditions.

Strings

To give the *condition* section a meaning you will also need a *strings* section. The strings sections is where you can define the strings that will be looked for in the file. Let's look at an easy example.

```

rule vendor
{
strings:
$text_string1 = "Vendor name" wide
$text_string2 = "Alias name" wide
condition:
$text_string1 or $text_string2
}

```

The rule shown above is named `vendor` and looks for the strings “Vendor name” and “Alias name”. If either of those strings is found, then the result of the rule is *true*.

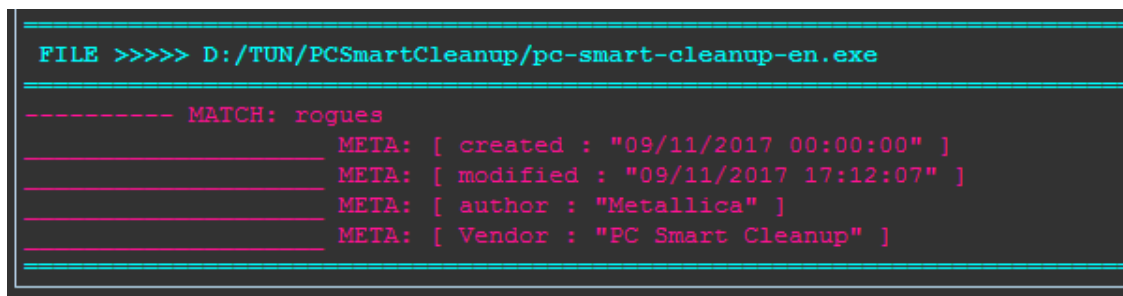
There are several types of strings you can look for:

- Hexadecimal, in combination with wild-cards, jumps, and alternatives.
- Text strings, with modifiers: `nocase`, `fullword`, `wide`, and `ascii`.
- Regular expressions, with the same modifiers as text strings.

There are many more advanced conditions you can use, but they are outside the scope of this post. If you would like to know more you can find it in the YARA documentation.

Metadata

Metadata can be added to help identify the files that were picked up by a certain rule. The metadata identifiers are always followed by an equal sign and the set value. The assigned values can be strings, integers, or a Boolean value. Note that identifier/value pairs defined in the metadata section can’t be used in the condition section, their only purpose is to store additional information about the rule.



```

FILE >>>> D:/TUN/PCSmartCleanup/pc-smart-cleanup-en.exe

----- MATCH: rogues
META: [ created : "09/11/2017 00:00:00" ]
META: [ modified : "09/11/2017 17:12:07" ]
META: [ author : "Metallica" ]
META: [ Vendor : "PC Smart Cleanup" ]

```

Fig 12:- Yara Scan and condition match

2.5 Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

Python Features

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below:

- It supports functional and structured programming methods as well as OOP.

- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- It supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

2.6 Machine Learning Libraries

2.6.1 Numpy

When starting to deal with the scientific task in Python, one inevitably comes for help to Python's SciPy Stack, which is a collection of software specifically designed for scientific computing in Python (do not confuse with SciPy library, which is part of this stack, and the community around this stack). This way we want to start with a look at it. However, the stack is pretty vast, there is more than a dozen of libraries in it, and we want to put a focal point on the core packages (particularly the most essential ones).

The most fundamental package, around which the scientific computation stack is built, is NumPy (stands for Numerical Python). It provides an abundance of useful features for operations on n-arrays and matrices in Python. The library provides vectorization of mathematical operations on the NumPy array type, which ameliorates performance and accordingly speeds up the execution.

2.6.2 Pandas

Pandas is a Python module that contains high-level data structures and tools designed for fast and easy data analysis operations.

Pandas is built on NumPy and makes it easy to use in NumPy-centric applications, such as data structures with labelled axes. Explicit data alignment prevents common errors that result from misaligned data coming in from different sources. It is also easy to handle missing data using Python. Pandas is the best tool for doing data munging.

2.6.3 Scikit-learn

Scikit-learn is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support vector machines, random forests, gradient boosting, k-means and DBSCAN, and is designed to interoperate with the Python numerical and scientific libraries NumPy and SciPy.

2.6.4 Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shell, the jupyter notebook, web application servers, and four graphical user interface toolkits.

Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, errorcharts, scatterplots, etc., with just a few lines of code. For a sampling, see the screenshots, thumbnail gallery, and examples directory.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc, via an object oriented interface or via a set of functions familiar to MATLAB users.

2.6.5 Seaborn

Seaborn is a library for making statistical graphics in Python. It is built on top of matplotlib and closely integrated with pandas data structures.

Here is some of the functionality that seaborn offers:

- A dataset-oriented API for examining relationships between multiple variables
- Specialized support for using categorical variables to show observations or aggregate statistics
- Options for visualizing univariate or bivariate distributions and for comparing them between subsets of data
- Automatic estimation and plotting of linear regression models for different kinds dependent variables
- Convenient views onto the overall structure of complex datasets
- High-level abstractions for structuring multi-plot grids that let you easily build complex visualizations
- Concise control over matplotlib figure styling with several built-in themes
- Tools for choosing color palettes that faithfully reveal patterns in your data

Seaborn aims to make visualization a central part of exploring and understanding data. Its dataset-oriented plotting functions operate on dataframes and arrays containing whole datasets and internally perform the necessary semantic mapping and statistical aggregation to produce informative plots.

2.7 Deep Learning Libraries

2.7.1 Keras

Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation. *Being able to go from idea to result as fast as possible is key to doing good research.*

TensorFlow 2.0 is an end-to-end, open-source machine learning platform. You can think of it as an infrastructure layer for differentiable programming. It combines three key abilities:

- Efficiently executing low-level tensor operations on CPU, GPU, or TPU.
- Computing the gradient of arbitrary differentiable expressions.
- Scaling computation to many devices (e.g. the Summit supercomputer at Oak Ridge National Lab, which spans 27,000 GPUs).
- Exporting programs ("graphs") to external runtimes such as servers, browsers, mobile and embedded devices.

Keras is the high-level API of TensorFlow 2.0: an approachable, highly-productive interface for solving machine learning problems, with a focus on modern deep learning. It provides essential abstractions and building blocks for developing and shipping machine learning solutions with high iteration velocity.

Keras empowers engineers and researchers to take full advantage of the scalability and cross-platform capabilities of TensorFlow 2.0: you can run Keras on TPU or on large clusters of GPUs, and you can export your Keras models to run in the browser or on a mobile device.

Installation & compatibility

Keras comes packaged with TensorFlow 2.0 as `tensorflow.keras`. To start using Keras, simply install TensorFlow 2.0.

Keras/TensorFlow is compatible with:

- Python 3.5–3.7
- Ubuntu 16.04 or later
- Windows 7 or later
- macOS 10.12.6 (Sierra) or later.
-

2.7.2 TensorFlow

Whether you're an expert or a beginner, TensorFlow is an end-to-end platform that makes it easy for you to build and deploy ML models. TensorFlow also supports an ecosystem of powerful add-on libraries and models to experiment with, including Ragged Tensors, TensorFlow Probability, Tensor2Tensor and BERT.

Tensorflow has the following features: -

- **Easy model building**

TensorFlow offers multiple levels of abstraction so you can choose the right one for your needs. Build and train models by using the high-level Keras API, which makes getting started with TensorFlow and machine learning easy.

If you need more flexibility, eager execution allows for immediate iteration and intuitive debugging. For large ML training tasks, use the Distribution Strategy API for distributed training on different hardware configurations without changing the model definition.

- **Robust ML production anywhere**

TensorFlow has always provided a direct path to production. Whether it's on servers, edge devices, or the web, TensorFlow lets you train and deploy your model easily, no matter what language or platform you use.

Use TensorFlow Extended (TFX) if you need a full production ML pipeline. For running inference on mobile and edge devices, use TensorFlow Lite. Train and deploy models in JavaScript environments using TensorFlow.js.

- **Powerful experimentation for research**

Build and train state-of-the-art models without sacrificing speed or performance. TensorFlow gives you the flexibility and control with features like the Keras Functional API and Model Subclassing API for creation of complex topologies. For easy prototyping and fast debugging, use eager execution.

CHAPTER 3: PROJECT DESIGN

The Project flow can be classified into 4 separate phases:

1. LAB SETUP
2. SAMPLE GENERATION
3. MEMORY ANALYSIS
4. ARTIFICIAL INTELLIGENCE

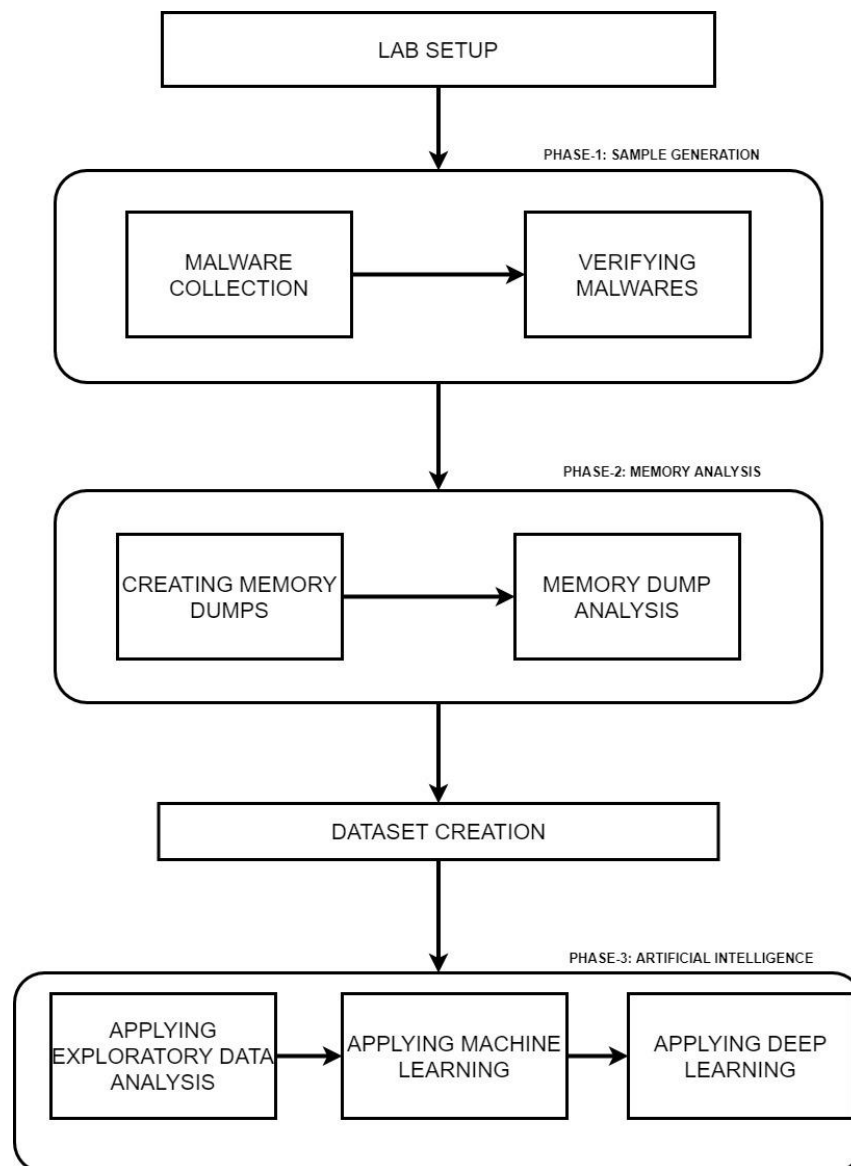


Fig 13: - Flow of Code of Project

PHASE 1: - LAB SETUP

To analyze malware, there is a need to setup a complete LAB including hypervisors, malware analysis tools, forensics tools etc.

The following **LABS** were created:-

1. MALWARE ANALYSIS LAB SETUP:

- VMware machine with WINDOWS 10(trial)
- Installation of FLARE-VM over Windows OS

2. MEMORY FORENSICS LAB SETUP:

- VMware machine with UBUNTU 18.04
- Installation of CUCKOO Sandbox
- Installation of Virtual box machine with Windows 10 inside VM machine
- Installation of Volatility
- Installation of Virtual box Ubuntu Server with Inetsim

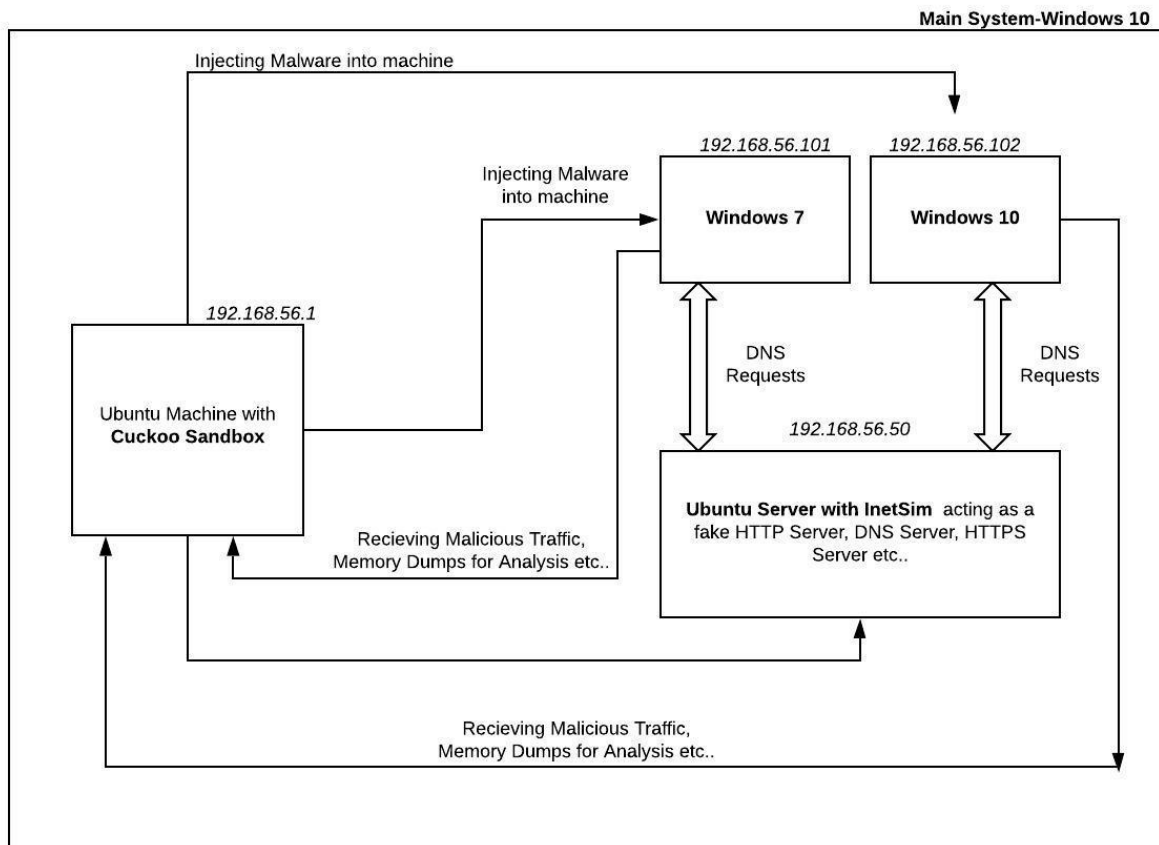


Fig 14:- Memory Forensics Lab Setup

PHASE 2: - SAMPLE GENERATION

To collect live malwares, many public repository websites were used. Some of the repositories were:-

1. Any.apprun
2. theZOO
3. <https://github.com/Endermanch/MalwareDatabase>
4. malware-samples@fabrimagic

Even though the samples were collected, still each of the sample needed to be verified by running it in a sandboxed environment. For initial analysis and dataset creation process only **RANSOMWARES** were considered.

To verify each malware each was executed within a virtual machine, with guest operating system as **Windows 10 x64, Build version 18362**.(Windows Developer trial)

The list of **RANSOMWARES** used are attached in **APPENDIX-A**.

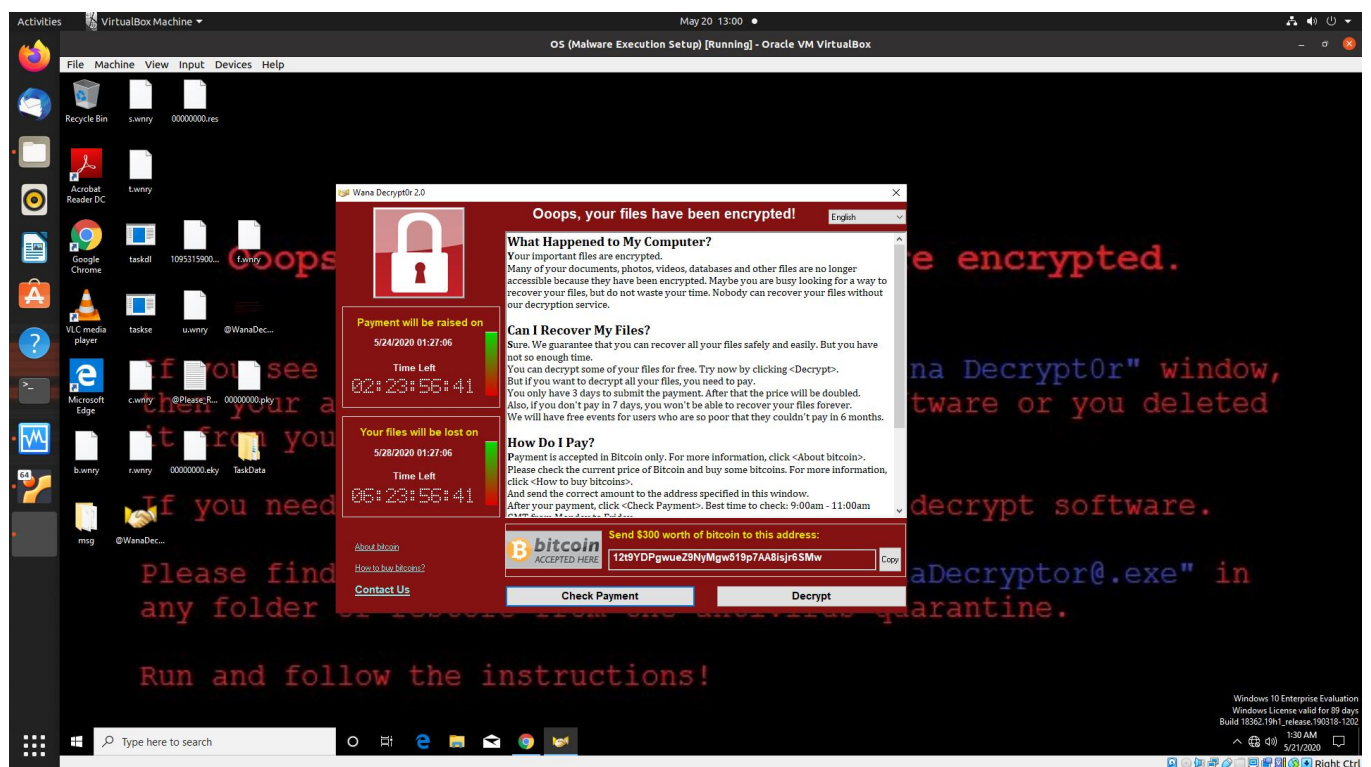


Fig 15:- Example of WannaCry Ransomware Executing in sandbox

PHASE 3: - MEMORY ANALYSIS

This Memory Analysis phase consists of two parts:

- a. Creating Memory Dumps
- b. Analyzing Memory Dumps

CREATING MEMORY DUMPS

To create a memory dump, of a malware executed sandbox these techniques are used:

- Use of Cuckoo Sandbox Automated Memory Dump feature.
- Use of VirtualBox command line feature.

COMMAND TO USE

```
VBoxManage debugvm <Virtual Machine name> dumpvmcore --  
filename=<filename>.elf
```

ANALYZING MEMORY DUMPS

To analyze memory dumps use of Volatility (memory forensics tool) is required. It is used to extract all crucial information about the systems. In our project, the following steps and commands were used to analyze memory.

STEPS:

1. Create the .elf into .raw file for proper analysis using the imagecopy command.(elf files have issue. See http://wiki.yobi.be/wiki/RAM_analysis for further details.)
2. Identify the build profile of the memory dump using imageinfo command.
3. Identify all the processes running in the system using the pslist command.
4. Identify all the hidden processes by running the psscan command and comparing the results with pslist command output.
5. With your Yara rules collection run the yarascan command to identify malicious memory samples.
6. Use this result for analysis.
7. Repeat the following steps for other memory dumps.

```
cuckoo@ubuntu:/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis$ vol.py -f WannaCry2.raw imageinfo
Volatility Foundation Volatility Framework 2.6.1
INFO : volatility.debug : Determining profile based on KDBG search...
      Suggested Profile(s) : Win10x64_18362
                             AS Layer1 : SkipDuplicatesAMD64PagedMemory (Kernel AS)
                             AS Layer2 : FileAddressSpace (/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis/WannaCry2.raw)
                             PAE type : No PAE
                             DTB : 0x1aa002L
                             KDBG : 0xf8053b8265e0L
      Number of Processors : 4
      Image Type (Service Pack) : 0
      KPCR for CPU 0 : 0xfffff8053855b000L
      KPCR for CPU 1 : 0xfffff8180b5fc0000L
      KPCR for CPU 2 : 0xfffff8180b6021000L
      KPCR for CPU 3 : 0xfffff8180b5be3000L
      KUSER_SHARED_DATA : 0xfffff78000000000L
      Image date and time : 2020-05-10 09:31:13 UTC+0000
      Image local date and time : 2020-05-10 15:01:13 +0530
cuckoo@ubuntu:/media/cuckoo/Seagate Backup Plus Drive/Major Project/Ransomware Analysis$
```

Fig 16: - Identifying Build profile of the Memory Dump.

```

Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x00879f28 74 61 73 6b 73 65 2e 65 78 65 00 00 40 57 61 6e taskse.exe..@Wan
0x00879f38 61 44 65 63 72 79 70 74 6f 72 40 2e 65 78 65 00 aDecryptor@.exe.
0x00879f48 74 61 73 6b 73 63 68 65 2e 65 78 65 00 00 00 00 tasksche.exe....
0x00879f58 25 00 73 00 5c 00 25 00 73 00 5c 00 25 00 73 00 %s.\.s.\.s.\.s.
0x00879f68 00 00 00 00 25 00 73 00 5c 00 2a 00 2e 00 2a 00 ....%s.\.*...*.
0x00879f78 00 00 00 00 40 57 61 6e 61 44 65 63 72 79 70 74 ....@WanaDecrypt
0x00879f88 6f 72 40 2e 65 78 65 2e 6c 6e 6b 00 5c 00 00 00 or@.exe.lnk.\...
0x00879f98 40 65 63 68 6f 20 6f 66 66 0d 0a 65 63 68 6f 20 @echo.off..echo.
0x00879fa8 53 45 54 20 6f 77 20 3d 20 57 53 63 72 69 70 74 SET.ow.=.WScript
0x00879fb8 2e 43 72 65 61 74 65 4f 62 6a 65 63 74 28 22 57 .CreateObject("W
0x00879fc8 53 63 72 69 70 74 2e 53 68 65 6c 6c 22 29 3e 20 Script.Shell")>.
0x00879fd8 6d 2e 76 62 73 0d 0a 65 63 68 6f 20 53 45 54 20 m.vbs..echo.SET.
0x00879fe8 6f 6d 20 3d 20 6f 77 2e 43 72 65 61 74 65 53 68 om.=.ow.CreateSh
0x00879ff8 6f 72 74 63 75 74 28 22 25 73 25 73 22 29 3e 3e ortcut("%s%s")>>
0x0087a008 20 6d 2e 76 62 73 0d 0a 65 63 68 6f 20 6f 6d 2e .m.vbs..echo.om.
0x0087a018 54 61 72 67 65 74 50 61 74 68 20 3d 20 22 25 73 TargetPath.=."%s

Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x0087a0a8 72 2e 77 6e 72 79 00 00 62 2e 77 6e 72 79 00 00 r.wnry..b.wnry..
0x0087a0b8 61 74 74 72 69 62 20 2b 68 20 2b 73 20 25 43 3a attrib.+h.+s.%C:
0x0087a0c8 5c 25 73 00 24 52 45 43 59 43 4c 45 00 00 00 00 \%. $RECYCLE....
0x0087a0d8 25 00 43 00 3a 00 5c 00 25 00 73 00 00 00 00 00 %C:.\.s.....
0x0087a0e8 24 00 52 00 45 00 43 00 59 00 43 00 4c 00 45 00 $.R.E.C.Y.C.L.E.
0x0087a0f8 00 00 00 00 25 00 73 00 5c 00 68 00 69 00 62 00 ....%s.\.h.i.b.
0x0087a108 73 00 79 00 73 00 25 00 73 00 00 00 20 00 3a 00 s.y.s.%s.....:
0x0087a118 5c 00 00 00 74 61 73 6b 64 6c 2e 65 78 65 00 00 \...taskdl.exe..
0x0087a128 25 73 0a 00 66 2e 77 6e 72 79 00 00 61 74 00 00 %s..f.wnry..at..
0x0087a138 63 6d 64 2e 65 78 65 20 2f 63 20 73 74 61 72 74 cmd.exe./c.start
0x0087a148 20 2f 62 20 25 73 20 76 73 00 00 00 25 73 20 63 ./b.%s.vs...%s.c
0x0087a158 6f 00 00 00 74 61 73 6b 6b 69 6c 6c 2e 65 78 65 o...taskkill.exe
0x0087a168 20 2f 66 20 2f 69 6d 20 6d 79 73 71 6c 64 2e 65 ./f./im.mysql.d.e
0x0087a178 78 65 00 00 74 61 73 6b 6b 69 6c 6c 2e 65 78 65 xe..taskkill.exe
0x0087a188 20 2f 66 20 2f 69 6d 20 73 71 6c 77 72 69 74 65 ./f./im.sqlwrite
0x0087a198 72 2e 65 78 65 00 00 00 74 61 73 6b 6b 69 6c 6c r.exe...taskkill

Rule: WannaDecryptor
Owner: Process WannaCry2.exe Pid 8496
0x0087a074 75 2e 77 6e 72 79 00 00 25 2e 31 66 20 42 54 43 u.wnry..%.1f.BTC
0x0087a084 00 00 00 00 24 25 64 20 77 6f 72 74 68 20 6f 66 ....%$d.worth.of
0x0087a094 20 62 69 74 63 6f 69 6e 00 00 00 00 77 00 62 00 .bitcoin....w.b.
0x0087a0a4 00 00 00 00 72 2e 77 6e 72 79 00 00 62 2e 77 6e ....r.wnry..b.wn
0x0087a0b4 72 79 00 00 61 74 74 72 69 62 20 2b 68 20 2b 73 ry..attrib.+h.+s
0x0087a0c4 20 25 43 3a 5c 25 73 00 24 52 45 43 59 43 4c 45 .%C:\%. $RECYCLE
0x0087a0d4 00 00 00 00 25 00 43 00 3a 00 5c 00 25 00 73 00 ....%C:.\.s.
0x0087a0e4 00 00 00 00 24 00 52 00 45 00 43 00 59 00 43 00 ....$.R.E.C.Y.C.
0x0087a0f4 4c 00 45 00 00 00 00 00 25 00 73 00 5c 00 68 00 L.E.....%s.\.h.
0x0087a104 69 00 62 00 73 00 79 00 73 00 25 00 73 00 00 00 i.b.s.y.s.%s....
0x0087a114 20 00 3a 00 5c 00 00 00 74 61 73 6b 64 6c 2e 65 ...\.taskdl.e
0x0087a124 78 65 00 00 25 73 0a 00 66 2e 77 6e 72 79 00 00 xe..%s..f.wnry..
0x0087a134 61 74 00 00 63 6d 64 2e 65 78 65 20 2f 63 20 73 at..cmd.exe./c.s
0x0087a144 74 61 72 74 20 2f 62 20 25 73 20 76 73 00 00 00 tart./b.%s.vs...
0x0087a154 25 73 20 63 6f 00 00 00 74 61 73 6b 6b 69 6c 6c %s.co...taskkill
0x0087a164 2e 65 78 65 20 2f 66 20 2f 69 6d 20 6d 79 73 71 .exe./f./im.mvsg

```

Fig 17:- Yarascan Result using Volatility

YARA RULES

The yara rules are collection of signatures that have been analyzed by malware analysts, that are used to identify malwares. These rules are certain conditions specific for each malware and help to identify it. Use of these rules have been done by majorly every endpoint security company, such as Virustotal, mcafee, palo Alto etc.

CREATING A DATASET

After creating scans and storing the results, the need is to model these scan results into a dataset. To model these results into dataset, use the following flowchart.

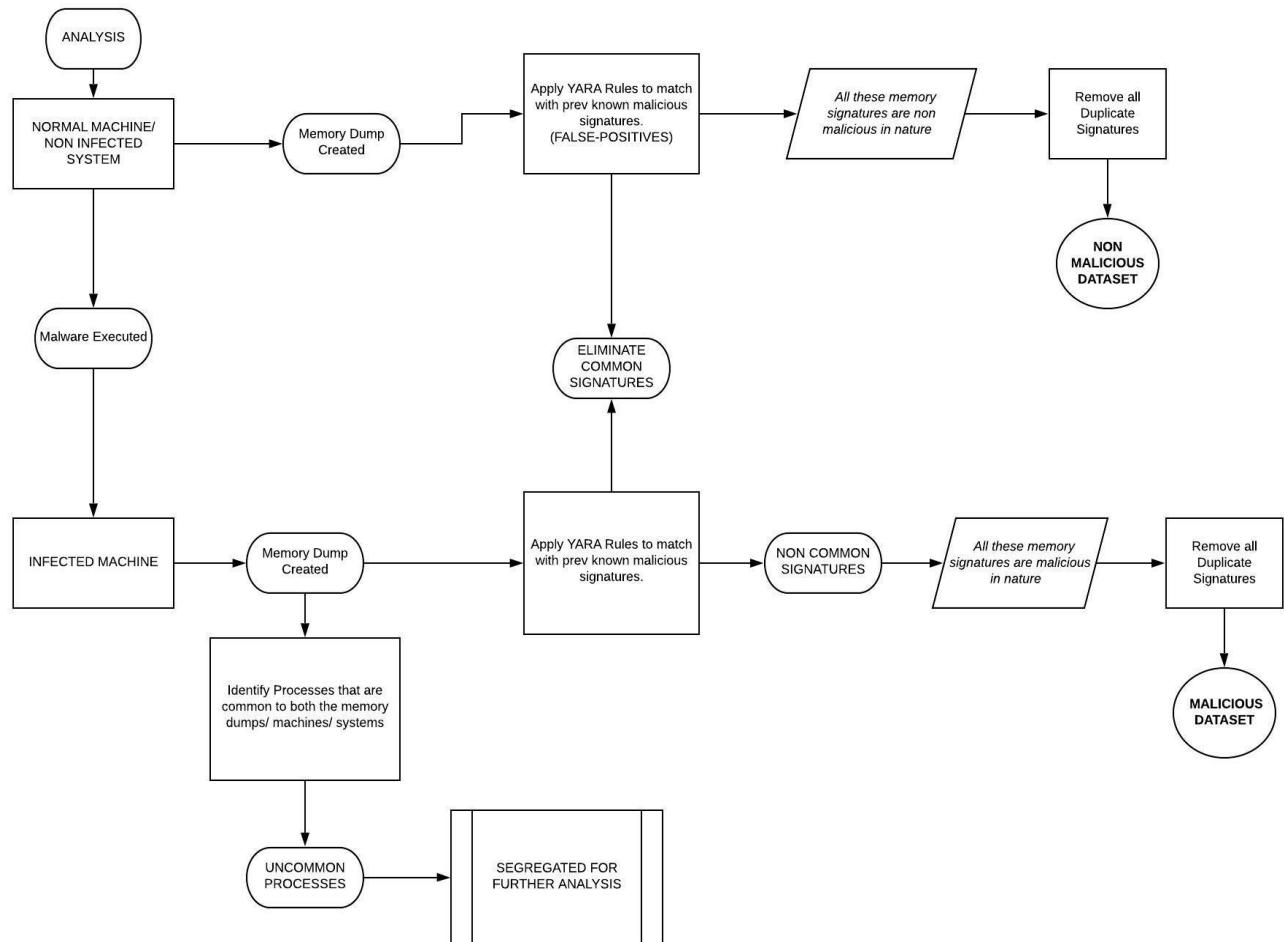


Fig 18: - Process of analysing malware and creating a dataset.

STEPS:-

1. Create Yarascan results of each malware using volatility
2. Remove common signatures that it has by matching it with ***standard benign samples***
3. Append these rules into a dataframe.
4. Keep record of benign and malicious samples by a creating a **TARGET** column

PHASE 4: - USING ARTIFICIAL INTELLIGENCE TO PREDICT MALWARE

After the creation of Dataset, it was time to create the dataset. This involved steps such as:

- a. Data Pre-Processing
 - a. Merging data from different Scan Results
 - b. Handling all Duplicated Values
 - c. Scaling Data: - Dividing data values by 255(16*16 memory samples produced) to scale data between 0 and 1.
 - d. Splitting data into Training and Testing
- b. Explortary Data Analysis

The Heat map doesn't produce much result. It depicts how features between the range [13,104] show slight correlation. This could help to reduce dimensions.

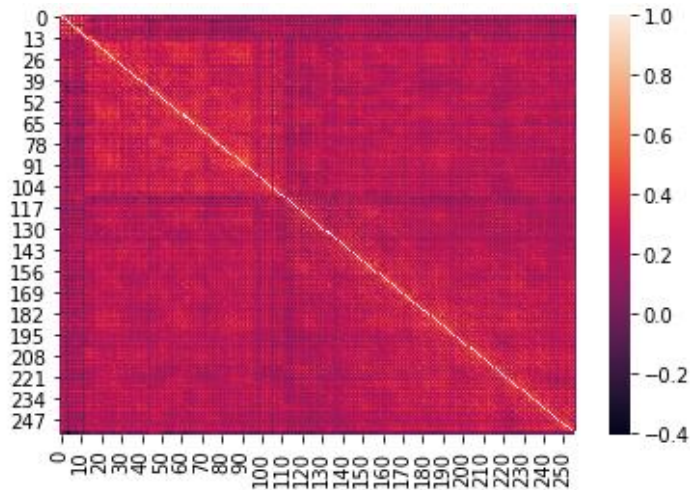


Fig 19:- Heat Map showing data correlation between features

The Count plot of Target Column shows the number of samples of each class.

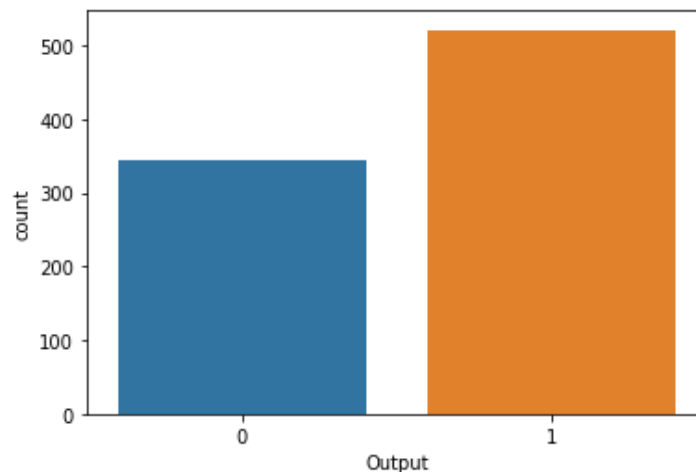


Fig 20:- Distribution of target column

On Applying PCA and TSNE to reduce dimensions to visualize each the samples, it can be conclusively said that samples tend to form compact clusters.

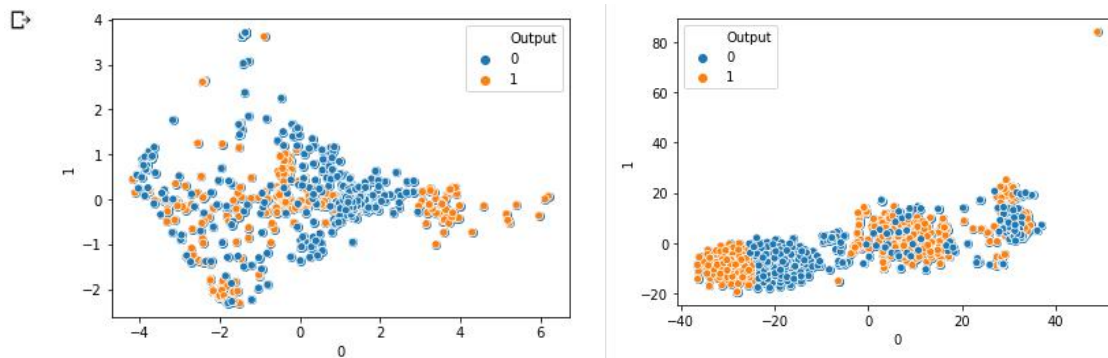


Fig 21: - Reduced Information Distribution of Malware and Benign Samples

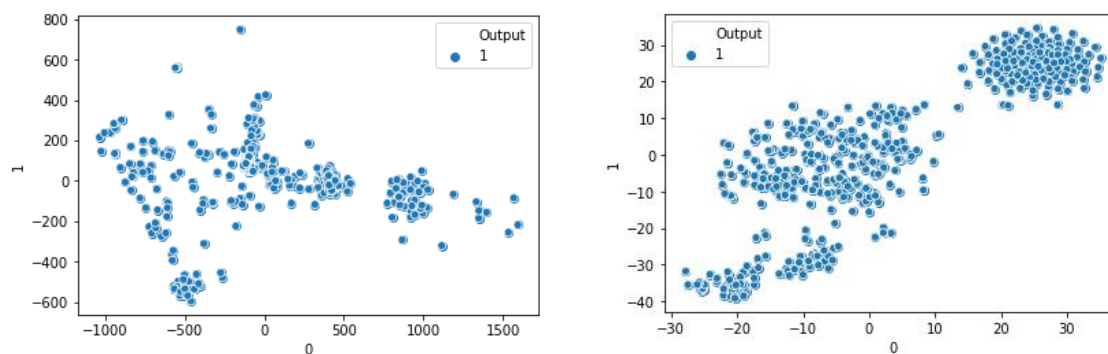


Fig 22: - Reduced Information Distribution of Malware samples only

This helps to identify that classification of such compact clusters, needs a highly refined and complex decision boundary/ function.

c. Applying Machine Learning Algorithms and Deep learning Algorithms

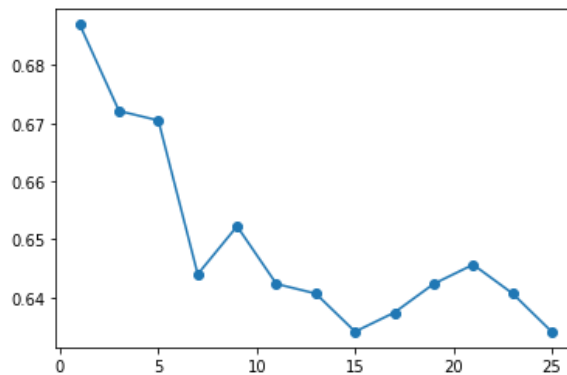
- On Applying various Machine Learning Algorithms, the metrics accuracy, precision, recall and F-1 Score were computed which are reported. The detailed discussion on results are discussed in the next chapter.
 - Classical supervised learning algorithms seem to perform poorly.
 - Ensemble techniques seem to perform better on the dataset.
 - High Training Score is obtained
- The Deep learning algorithm used currently is ANN(artificial neural network). It consists of dense and dropout layers. The model needs a better model majorly a model that can remember the previous value, such as an RNN or LSTM etc.

- The following example of KNN algorithm proves are PCA and TSNE graphs/plots. As the clusters were compact and in nature, and malware and benign samples were close to each other, the value of K had to be low. The score/accuracy should decrease with increase in K-nearest neighbours.

```
KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                    metric_params=None, n_jobs=None, n_neighbors=3, p=2,
                    weights='uniform')
```

Score of KNN on Training Data: 0.7682119205298014

Score of KNN on Testing Score: 0.6923076923076923



Best K-Nearest Neighbour Value: 1

Fig 23:- Using KNN with dataset and mapping score with K using cross validation

CHAPTER 4: RESULTS AND CONCLUSION

From the Machine Learning Analysis we could clearly see that clusters were being formed by malicious samples and benign samples. Thus processes exhibiting similar type of behavior in system have similar memory values in memory. Using this when we applied machine learning and deep learning to dataset we got the following result:

Name of Classifier	Accuracy	Precision	Recall	F1 Score
KNN Classifier	0.692	0.71	0.69	0.70
SVM Classifier	0.769	0.77	0.77	0.77
Naïve Bayes Classifier	0.657	0.66	0.66	0.66
Logistic Regression	0.730	0.73	0.73	0.73
Decision Tree Classifier	0.80	0.81	0.81	0.81
Random Forest Classifier	0.857	0.86	0.86	0.86
Gradient Boosting Classifier	0.85	0.85	0.85	0.85
Bagging Classifier	0.857	0.86	0.86	0.86
AdaBoost Classifier	0.792	0.79	0.79	0.79

Table-1:- Values of different parameters for all classifiers after classification

On Applying Machine Learning, on the memory values the model seems to perform well. It has an average training score and is able to predict the new malwares from the previously learnt malwares. The dataset needs to be increased for it to have a higher accuracy.

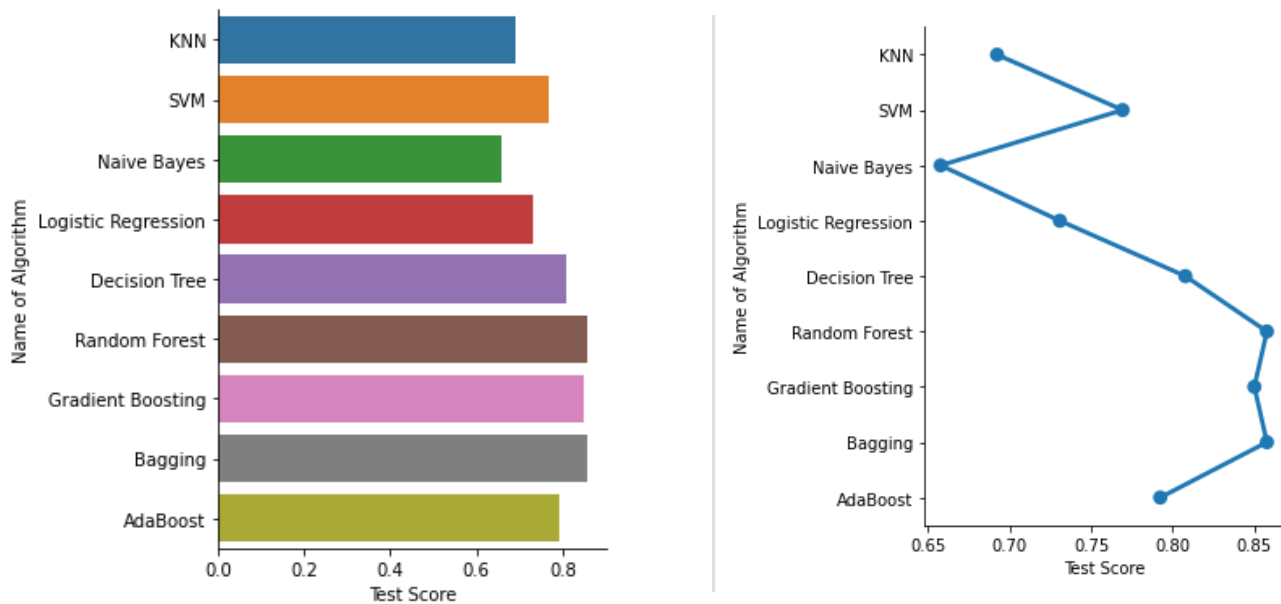


Fig 24:- Comparing Testing Scores of Dataset

On Applying Machine Learning, on the memory values the model seems to perform well. It has a high training score because which it should as it needs to remember all the values of the malwares given to it.

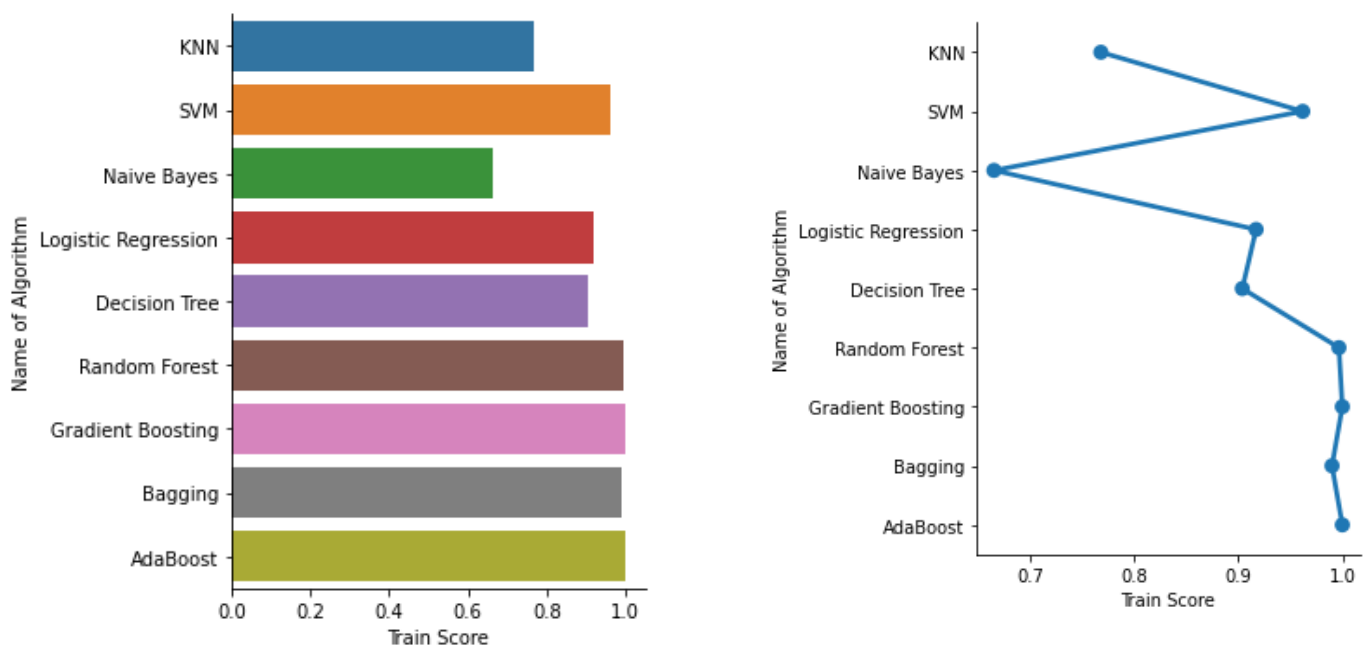


Fig 25:- Comparing Training Scores of Dataset

CHAPTER 5: FUTURE SCOPE

This project still currently needs a lot of work. So the following works will be executed after the current submission of the project:-

1. The list of YARA rules collected is still very small. The rule set needs to be expanded to include other open source rules from Alienvault, McAfee, Palo Alto etc.
2. This technique if is to be utilized as a full scale antivirus needs an interface.
3. The technique can be upgraded if the use of rules is eliminated and in depth analysis of malwares is done and how they affect the memory hexadecimal values.
4. Operating System concepts of pages, Virtual Addresses needs to be applied.
5. Use of GAN with this technique is pending.
6. Use of other plugins of Volatility for analysis could be done
7. Automating the entire process.

CHAPTER 6: REFERENCES

1. https://en.wikipedia.org/wiki/Category:Virtualization_software
2. <https://www.redhat.com/en/topics/virtualization/what-is-a-hypervisor>
3. <https://digitalguardian.com/blog/what-malware-analysis-defining-and-outlining-process-malware-analysis>
4. <https://www.crowdstrike.com/epp-101/malware-analysis/>
5. <https://www.virtualbox.org/>
6. https://www.virtualbox.org/wiki/End-user_documentation
7. <https://www.vmware.com/products/player/faqs.html>
8. <https://digitalguardian.com/blog/what-are-memory-forensics-definition-memory-forensics>
9. <https://github.com/volatilityfoundation/volatility/wiki/Command-Reference-Mal>
10. <https://github.com/volatilityfoundation/volatility>
11. <https://github.com/volatilityfoundation/volatility/wiki/Command-Reference>
12. <https://www.youtube.com/watch?v=PDnFJe9qrLA>
13. <https://seaborn.pydata.org/introduction.html>
14. <https://keras.io/about/>

APPENDIX-A

List of Malwares that were used:-

1. Annabelle.exe
2. Blue Howl.exe
3. Ceber.exe
4. Cerber1.exe(another version)
5. GandCrab.exe
6. Jigsaw1.exe
7. Jigsaw2.exe(another version)
8. TeslaCrypt1.exe
9. Vipasana1.exe
10. ViraLock.exe
11. WannaCry1.exe
12. WannaCry2.exe(another version)